Autoría: [Nombre y apellidos de los integrantes]

Reto 1 · Abstracción

Especificación del TDA Juego de la Vida

Estructuras de Datos · Universidad de Granada

1. Tipo de dato y estado abstracto

El TDA representa un patrón finito del Juego de la Vida y su evolución en un plano sin límites prefijados. Cada celda tiene uno de dos estados: viva o muerta. Fuera del rectángulo representado, todas las celdas están muertas. Los bordes no se conectan entre sí.

Su estado abstracto es (P, g): P es el patrón actual y g es el número de generaciones transcurridas desde la última carga correcta. P se describe mediante una matriz rectangular de estados, de F filas y C columnas. La población V es el número de celdas vivas de P.

Invariantes del TDA

- Patrón no vacío: F y C son positivos; cada uno de los cuatro bordes contiene al menos una celda viva. Por tanto, P ocupa el rectángulo mínimo que contiene todas las celdas vivas.
- Patrón vacío: F = C = V = 0 y no hay celdas almacenadas. Esta convención resuelve la extinción, que el enunciado no define, y es la única excepción a la condición de bordes vivos.
- Siempre se cumple $0 \leq V \leq F \cdot C$ y $g \geq 0$. Todos los observadores describen la misma generación completa.

Las coordenadas públicas son locales: la esquina superior izquierda del rectángulo actual es (0, 0). Al recortarlo pueden cambiar las coordenadas de una celda. No se conserva una posición absoluta, porque los ficheros solo describen el patrón y las reglas son invariantes frente a traslaciones.

2. Regla de evolución

Las vecinas de una celda son las ocho posiciones contiguas, incluidas las diagonales, excluyendo la propia celda. Sea k el número de vecinas vivas en la generación anterior, con $0 \leq k \leq 8$. La celda queda viva si k = 3, o si ya estaba viva y k = 2; en cualquier otro caso queda muerta.

> La transición es simultánea: todas las decisiones utilizan exclusivamente la generación anterior. También se consideran las posiciones de la franja exterior de una celda, donde pueden producirse nacimientos.

3. Interfaz pública: creación y consultas

Los contratos siguientes definen operaciones abstractas, sin cuerpos de funciones ni algoritmos. P_0 y g_0 designan el estado anterior a una operación. Las precondiciones deben cumplirse al invocarla; los errores de acceso, formato o recursos se comunican explícitamente.

crear · Constructor por defecto

Entradas: Ninguna.

Precondición: Ninguna.

Resultado y postcondición: Un objeto válido con patrón vacío, dimensiones 0 × 0, población 0 y contador g = 0. Si no puede construirse por falta de recursos, se comunica el error y no se obtiene un objeto.

cargarPatron · Sustituir el patrón desde un fichero

Entrada: Nombre del fichero, una cadena de texto.

Precondición: El objeto satisface los invariantes.

Resultado: Éxito, o error de nombre vacío, acceso, formato o recursos.

Postcondición si tiene éxito: P coincide con el patrón descrito en el fichero; F, C y V son coherentes con él; g = 0. Se reemplaza el estado anterior, sin acumular generaciones ni celdas.

Postcondición si falla: $P = P_0$ y $g = g_0$: el objeto conserva íntegramente su estado anterior.

La lectura es estricta: valida las dimensiones, la cantidad y longitud de las filas, los caracteres y la condición de rectángulo mínimo. Rechaza bordes totalmente muertos en patrones no vacíos; no corrige silenciosamente una entrada fuera del formato. El formato completo se fija en el apartado 6.

Observadores

Precondición común: objeto válido. Postcondición común: no modifican P ni g, y no proporcionan acceso modificable a la representación interna.

Operación / entradas	Valor devuelto
obtenerDimensiones Sin entradas	El par de enteros no negativos (F, C); (0, 0) para el vacío.
obtenerNumeroCeldasVivas Sin entradas	El entero no negativo V: número exacto de celdas vivas del patrón actual.
obtenerIteraciones Sin entradas	El entero no negativo g: generaciones completadas desde la última carga correcta.
obtenerCelda Fila f y columna c, enteras	El estado viva o muerta de P(f, c). Precondición adicional: $0 \leq f < F$ y $0 \leq c < C$. No es invocable sobre el patrón vacío.

4. Interfaz pública: evolución y escritura

avanzarGeneracion · Una transición completa

Entradas: Ninguna.

Precondición: Objeto válido.

Resultado: Éxito, o error de recursos o de capacidad de representación.

Postcondición si tiene éxito: P es el resultado de aplicar simultáneamente la regla del apartado 2 a P_0, incluido su exterior. Se normaliza al rectángulo mínimo; F, C y V corresponden al nuevo patrón; $g = g_0 + 1$.

Postcondición si falla: $P = P_0$ y $g = g_0$. No queda una generación parcialmente actualizada.

Si P_0 está vacío, P sigue vacío y g aumenta en uno. Si desaparecen todas las celdas, el nuevo estado tiene dimensiones 0×0. El rectángulo puede crecer o disminuir; no se descartan nacimientos por falta de espacio prefijado.

simular · Evolucionar varias generaciones

Entrada: Un entero n: número de generaciones adicionales.

Precondición: Objeto válido y $n \geq 0$.

Resultado: Éxito o error, y un entero r que indica cuántas generaciones nuevas se completaron.

Postcondición si tiene éxito: $r = n$; P es la evolución de P_0 tras exactamente n transiciones; $g = g_0 + n$. Si $n = 0$, el objeto no cambia. Los estados estables o vacíos también contabilizan las n generaciones lógicas.

Postcondición si falla: $0 \leq r < n$; P es la evolución exacta de P_0 tras r transiciones y $g = g_0 + r$. Se conserva la última generación completa y válida; se comunica la causa del error.

guardarPatron · Serializar el patrón actual

Entrada: Nombre del fichero de destino, una cadena de texto.

Precondición: Objeto válido.

Resultado: Guardado, omitido o error de acceso/escritura.

Postcondición: El objeto no cambia. Si el nombre está vacío, se omite la escritura y no se crea ni modifica ningún fichero. Si se guarda correctamente, el destino contiene exactamente P en el formato del apartado 6, sustituyendo su contenido anterior si existía.

Una escritura fallida no garantiza el contenido del destino, que puede quedar incompleto, pero conserva el estado del TDA. No se serializan g ni el tiempo: una carga posterior del patrón guardado lo recupera con contador cero.

5. Encapsulación y eficiencia

Representación privada prevista

Una matriz rectangular densa y dinámica almacena un valor por celda, sin compresión. Puede materializarse mediante vectores de igual longitud. Se acompañará de contadores escalares para dimensiones, población e iteraciones. La capacidad reservada es un detalle privado y puede incluir márgenes muertos reutilizables.

La función de abstracción toma la región activa de esa matriz como P. Su invariante exige filas de igual longitud, estados válidos, dimensiones y población coherentes y una región activa mínima, o vacía. Cualquier zona reservada exterior representa celdas muertas y no se muestra en la salida.

Se prevé una segunda matriz rectangular para separar la generación anterior de la siguiente. La interfaz oculta la reserva de memoria, los márgenes, el recorte y el cambio de matrices. No se usan árboles, tablas hash, listas enlazadas ni representaciones comprimidas.

Responsabilidades auxiliares privadas

contarVecinosVivos(f, c): Pre: generación de referencia válida y coordenadas enteras locales, que pueden estar fuera de P. Devuelve un entero de 0 a 8, ambos incluidos: la suma de estados de las ocho vecinas, tomando como muerta cualquier posición exterior. Post: no modifica la generación.

ajustarRectangulo: Pre: una matriz rectangular candidata con estados válidos. Post: conserva exactamente sus celdas vivas y sus posiciones relativas, elimina únicamente filas y columnas exteriores totalmente muertas y actualiza dimensiones y población; si no hay vivas, produce 0 × 0. No cambia el contador de generaciones.

Estos auxiliares no son operaciones disponibles para el usuario del TDA. La ampliación de capacidad conserva el patrón y los contadores; solo añade espacio muerto de trabajo, sin constituir una generación.

Costes previstos, sin desarrollar algoritmos

Para un patrón no vacío de F × C, un paso solo necesita considerar el rectángulo (F + 2) × (C + 2): a mayor distancia no puede haber vecinas vivas. El tiempo previsto por generación y el espacio de trabajo son O((F + 2)(C + 2)). La población y los límites pueden obtenerse durante la construcción de la nueva generación.

Para n generaciones, el tiempo es la suma de los costes de sus áreas sucesivas; el espacio depende del mayor rectángulo utilizado, no de almacenar todo el historial. Las consultas de dimensiones, población e iteraciones son O(1) si los metadatos se mantienen coherentes. La lectura y la escritura son O(F · C) para patrones no vacíos.

> No existe un máximo fijo de filas o columnas impuesto por el diseño. Los límites físicos de memoria o de representación se comunican como errores, sin recortar células vivas ni alterar silenciosamente la evolución.

6. Formato y uso desde el programa cliente

Contrato del fichero

Primera línea: entero decimal F. Segunda línea: entero decimal C. Para un patrón no vacío, siguen exactamente F líneas, cada una con exactamente C caracteres: punto (.) para celda muerta y X mayúscula para celda viva. No se admiten espacios dentro de las filas, otros símbolos ni líneas de datos adicionales. Se acepta el fin de línea habitual LF o CRLF; el salto final es opcional.

F y C son positivos en patrones no vacíos, y cada borde contiene alguna X. Para el vacío se acuerda una extensión mínima del formato: dos líneas con 0 y 0, sin filas posteriores. Se rechazan dimensiones negativas y los casos 0 × C o F × 0 con la otra dimensión positiva.

Ejemplo válido: oscilador horizontal

```
1
3
XXX
```

Responsabilidades del programa cliente

El cliente obtiene el nombre del fichero inicial y el entero n, comprueba que n no sea negativo, carga el patrón y solicita simular(n). Mide el tiempo con un reloj externo al TDA; aquí se adopta el tiempo transcurrido entre el inicio de la carga y el fin de la simulación, excluyendo la interacción y el guardado.

Tras una simulación correcta, muestra exactamente estas cuatro líneas, en el orden exigido:

1. Número de iteraciones realizadas: obtenerIteraciones.
2. Número de celdas vivas del patrón final: obtenerNumeroCeldasVivas.
3. Dimensiones mínimas del patrón final: obtenerDimensiones.
4. Tiempo empleado, expresado en segundos.

Después solicita el nombre del fichero de salida y aplica guardarPatron; una cadena vacía omite el guardado. Si hay un error de carga o simulación, lo comunica y no presenta un resultado incompleto como si se hubiesen realizado todas las iteraciones solicitadas. La consola y el cronómetro no forman parte del TDA.

Ejemplos de cumplimiento de los contratos

Situación	Resultado esperado
Cualquier patrón, n = 0	Se conserva el patrón cargado; g = 0.
Bloque 2 × 2, todas vivas; n = 1	Mismo patrón; V = 4 y g = 1.
Patrón 1 × 3: XXX; n = 1	Patrón 3 × 1, tres X en vertical; V = 3. Hay nacimientos fuera del rectángulo inicial.
Una sola X; n = 1	Patrón vacío 0 × 0; V = 0 y g = 1; fichero final con dos líneas 0.

Base: enunciado reto1-2026.pdf y presentación reto1-presentacion.pdf. La entrega especifica el TDA; no incluye implementación del juego.